

Experiment Number: 01

Experiment Name: Highest Gross Salary.

Objectives:

- To calculate Gross Salary based on Basic Salary.
- To display the employee who has the highest Gross Salary.

Theory:

In this experiment we have used employee class with attributes id, name, basic salary and calculating gross salary. Here, gross salary is calculated using the formula:

$$\text{Gross Salary} = \text{Basic Salary} + 0.20 \times \text{Basic Salary} + 0.10 \times \text{Basic Salary}$$

Algorithm:

1. Start the program.
2. Create a class Employee with members for ID, Name, Basic Salary, and Gross Salary.
3. Define a function to calculate Gross Salary using the formula.
4. Take input of N employee's details.
5. Calculate Gross Salary for each employee.
6. Display that employee's ID, Name, and Gross Salary.
7. End the program.

Code:

```
rt here x *HighestGrossSalary.cpp.cpp x
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4  class Employee {
5  public:
6      int id;
7      string name;
8      double basicSalary;
9      double grossSalary;
10
11  void calculateGross() {
12      grossSalary = basicSalary + (0.20 * basicSalary) + (0.10 * basicSalary);
13  }
14 };
15 int main() {
16     int N;
17     cout << "Enter number of employees you want to calculate: ";
18     cin >> N;
19     Employee emp, highest;
20     highest.grossSalary = 0;
21     for (int i = 0; i < N; i++) {
22         cout << "\nEnter employee ID, Name, Basic Salary: ";
23         cin >> emp.id >> emp.name >> emp.basicSalary;
24         emp.calculateGross();
25         cout << endl;
26         if (emp.grossSalary > highest.grossSalary) {
27             highest = emp;
28         }
29     }
30     cout << "\nEmployee with Highest Gross Salary:\n";
31     cout << "ID: " << highest.id << endl;
32     cout << "Name: " << highest.name << endl;
33     cout << fixed << setprecision(2);
34     cout << "Gross Salary: " << highest.grossSalary << endl;
35     return 0;
```

Output:

```
"D:\Versity Work\BAUST CSE\ x + v
Enter number of employees you want to calculate: 2
Enter employee ID, Name, Basic Salary: 101 Sara 10000
Enter employee ID, Name, Basic Salary: 102 Rynn 15000
Employee with Highest Gross Salary:
ID: 102
Name: Rynn
Gross Salary: 19500.00
```

Conclusion:

In this program we have successfully calculated the Gross Salary using the given formula. We have compared the salaries and showed the highest Gross Salary and their id and names.

Experiment Number: 02

Experiment Name: Complex Number.

Objectives:

- To create a program that performs addition and subtraction of two complex numbers.
- To understand how this math can be done with OOPS in C++.

Theory:

A complex number is of the form $a + bi$, where a_1, a_2 are the real part and b_1, b_2 are the imaginary part. In C++, we have taken complex number using a class with two data members by defining functions for addition and subtraction, we have calculated complex numbers like mathematical objects.

Algorithm:

1. Start the program
2. Define a class Complex with real and imaginary parts.
3. Take input of two complex numbers from the user.
4. Use functions to perform addition and subtraction.
5. Display the result.
6. End the program.

Code:

```
1  #include <iostream>
2  using namespace std;
3  class Complex {
4  public:
5      int real;
6      int imag;
7  };
8  Complex add(Complex c1, Complex c2) {
9      Complex temp;
10     temp.real = c1.real + c2.real;
11     temp.imag = c1.imag + c2.imag;
12     return temp;
13 }
14 Complex sub(Complex c1, Complex c2) {
15     Complex temp;
16     temp.real = c1.real - c2.real;
17     temp.imag = c1.imag - c2.imag;
18     return temp;
19 }
20 int main() {
21     Complex c1, c2, sum, diff;
22     int a1, b1, a2, b2;
23     cout << "Taking input of 1st complex number: ";
24     cin >> a1 >> b1;
25     cout << "Now for 2nd complex number: ";
26     cin >> a2 >> b2;
27     c1.real = a1;
28     c1.imag = b1;
29     c2.real = a2;
30     c2.imag = b2;
31
32     sum = add(c1, c2);
33     diff = sub(c1, c2);
34     cout << "\nSum: " << sum.real << " " << sum.imag << "i" << endl;
35     cout << "Difference: " << diff.real << " " << diff.imag << "i" << endl;
36     return 0;
37 }
```

Output:

```
"D:\Versity Work\BAUST CSE\  +  v
Taking input of 1st complex number: 1 2
Now for 2nd complex number: 3 4

Sum: 4 6i
Difference: -2 -2i
```

Conclusion:

In this program we have successfully showed the concept of complex numbers in C++ using OOPS. By using a class to show the real and imaginary parts, and separate functions for addition and subtraction, the program shows with object-oriented programming how we can solve mathematical operations.

Experiment Number: 03

Experiment Name: Who is older teacher or student.

Objectives:

- To understand how to use constructor overloading.
- To understand how to use friend function to access data from another class.

Theory:

Friend functions allows us to access to private members of a class. By defining a friend function, we compare the ages of a student and a teacher, we can determine who is older even though age is private in both classes.

Algorithm:

1. Start the program.
2. Create class Student with private members.
3. Create class Teacher with private members.
4. Declare a friend function in both classes.
5. Find the age difference.
6. Create objects of Student and Teacher.
7. Call compare and show who is older.
8. End the program.

Code:

```
1  #include <iostream>
2  using namespace std;
3  class Teacher;
4  class Student{
5  private:
6      double id;
7      int age;
8      float cgpa;
9  public:
10     Student (double i, int a, float c){
11         id = i;
12         age = a;
13         cgpa = c;
14     }
15     friend int compare (Student s, Teacher t);
16 };
17 class Teacher{
18 private:
19     string department;
20     int age;
21     char ranked;
22 public:
23     Teacher (string d, int a, char r){
24         department = d;
25         age = a;
26         ranked = r;
27     }
28     friend int compare (Student s, Teacher t);
29 };
30 int compare (Student s, Teacher t){
31     return t.age - s.age;
32 }
33 int main(){
34     int difference;
35     Student s (1002, 22, 3.50);
36     Teacher t ("Math", 45, 'A');
37     difference = compare (s, t);
38
39     if (difference > 0) cout << "Teacher is older" << endl;
40     else if (difference == 0) cout << "The age of teacher and student is same" << endl;
41     else cout << "Student is older" << endl;
42     return 0;
43 }
```

Output:

```
"D:\Versity Work\BAUST CSE\  x + v
Teacher is older
Process returned 0 (0x0)    execution time : 0.059 s
Press any key to continue.
```

Conclusion:

In this program we have successfully showed who is older. We have used friend functions to compare the private age attribute of Student and Teacher. Since the teacher's age (45) is greater than the student's age (22), the program correctly showed "Teacher is older".

Experiment Number: 04

Experiment Name: Introduction to Inheritance and Friend function.

Objectives:

- To understand how Inheritance work.
- To understand how Friend function works.

Theory:

In OOP inheritance allows derived class to acquire the properties, data members and member functions of another class called base class. A friend function is a function that is not a member of a class, but it is given special permission to access the private and protected members of that class.

Code: 1

```
1  #include<iostream>
2  using namespace std;
3  class A{
4      int x;
5  public:
6      int y;
7      void setx(int a,int b){
8          x=a;
9          y=b;
10     }
11     void show(){
12         cout<<x<<y<<endl;
13     }
14 };
15 class B:public A{
16     int w;
17 public:
18     void sety(int a){
19         w=a;
20     }
21     void sshow(){
22         cout<<w<<y;
23     }
24 };
25 int main(){
26     A objA;
27     B objB;
28     objA.setx(1,2);
29     objB.sety(20);
30     objA.show();
31     objB.sshow();
32     return 0;
33 }
```

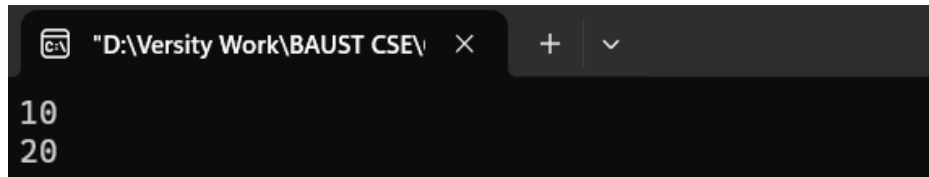
Output:

```
"D:\Versity Work\BAUST CSE\  x + v
12
200
Process returned 0 (0x0)    execution time : 0.071 s
```

Code: 2

```
1  #include<iostream>
2  using namespace std;
3  class Base{
4      int x;
5  public:
6      void setx(int a){
7          x=a;
8      }
9      void showx(){
10         cout<<x<<endl;
11     }
12 };
13 class Derived:public Base{
14     int y;
15 public:
16     void sety(int b){
17         y=b;
18     }
19     void showy(){
20         cout<<y<<endl;
21     }
22 };
23 int main(){
24     Derived obj;
25     obj.setx(10);
26     obj.sety(20);
27     obj.showx(); //10 output
28     obj.showy(); //20 output
29     return 0;
30 }
```

Output:



```
"D:\Versity Work\BAUST CSE\  X + v
10
20
```

Code: 3

```
1  #include<iostream>
2  using namespace std;
3  class Base{
4      int x;
5  public:
6      int retx(){
7          return x;
8      }
9      void setx(int a){
10         x=a;
11     }
12     void showx(){
13         cout<<x<<endl;
14     }
15 };
16 class Derived:public Base{
17     int y;
18 public:
19     void sety(int b){
20         y=b;
21     }
22     void showsun(){
23         cout<<retx()+y<<endl;
24     }
25     void showy(){
26         cout<<y<<endl;
27     }
28 };
29 int main(){
30     Derived obj;
31     obj.setx(10);
32     obj.sety(20);
33     obj.showsun();
34     obj.showx();
35     obj.showy();
36     return 0;
}
```

Output:



```
"D:\Versity Work\BAUST CSE\  X + v
30
10
20
Process returned 0 (0x0)  execution time : 0.064 s
```

Code: 4

```
1 | #include<iostream>
2 | using namespace std;
3 | class Base{
4 |     int x;
5 | public:
6 |     void setx(int a){
7 |         x=a;
8 |     }
9 |     void showx(){
10 |         cout<<x<<endl;
11 |     }
12 | };
13 | class Derived:private Base{
14 |     int y;
15 | public:
16 |     void sety(int b){
17 |         y=b;
18 |     }
19 |     void showy(){
20 |         cout<<y<<endl;
21 |     }
22 | };
23 | int main(){
24 |     Derived obj;
25 |     obj.setx(10); // setx is now private so it can't be accessed
26 |     obj.sety(20);
27 |     obj.showx();
28 |     obj.showy();
29 |     return 0;
30 | }
```

Output:

Line	Message
25	error: 'void Base::setx(int)' is inaccessible within this context
6	note: declared here

Conclusion:

In this experiment we have learned how to use Inheritance and Friend function, some codes have run successfully and some codes have showed some error. This helps us to do better in programing.